

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО

ОТЧЁТ
ПО ДИСЦИПЛИНЕ «БЭК-ЭНД РАЗРАБОТКА»
по теме:
Д35: РЕАЛИЗАЦИЯ МЕЖСЕРВИСНОГО ВЗАИМОДЕЙСТВИЯ
ПОСРЕДСТВОМ ОЧЕРЕДЕЙ СООБЩЕНИЙ

Проверил:

подпись, дата

Добряков Д. И.

Выполнил:

подпись, дата

Субботин Александр

1 Задача

- Подключить и настроить RabbitMQ.
- Реализовать межсервисное взаимодействие посредством RabbitMQ.

Стек: Go, Gin, RabbitMQ 3.13, amqp091-go, Docker Compose.

2 Ход работы

2.1 1. Выбор брокера сообщений

Выбран **RabbitMQ** (образ `rabbitmq:3.13-management-alpine`).
Management-плагин доступен на порту 15672 для мониторинга очередей.

2.2 2. Архитектура обмена событиями

Реализована модель **publish/subscribe** через **fanout exchange**:

- **Publisher** — `booking-service`: публикует события при создании бронирования и смене статуса.
- **Consumer** — `property-service`: потребляет события бронирований (для логирования, обновления статистики).

Exchange: `booking_events` (тип: `fanout`, `durable`) **Queue:**
`property_booking_events` (`durable`, привязана к exchange)

2.2.1 2.1 Формат события

```
{  
  "event_type": "booking.created",  
  "booking_id": "uuid",  
  "property_id": "uuid",  
  "tenant_id": "uuid",  
  "status": "pending",  
  "price_total": 25000.00  
}
```

Типы событий:

- `booking.created` — создано новое бронирование
- `booking.status_changed` — изменён статус (`approved`, `rejected`, `cancelled`, `completed`)

2.3 3. Реализация

2.3.1 3.1 Publisher (booking-service)

Файл: `internal/messaging/publisher.go`

- При старте сервиса устанавливается соединение с RabbitMQ.
- Объявляется exchange `booking_events` (fanout, durable).
- После создания бронирования (`Create`) и обновления статуса (`UpdateStatus`) публикуется событие.
- Если RabbitMQ недоступен — сервис работает без публикации (graceful degradation).

2.3.2 3.2 Consumer (property-service)

Файл: `internal/messaging/consumer.go`

- При старте сервиса устанавливается соединение и объявляется очередь `property_booking_events`.
- Очередь привязывается к exchange `booking_events`.
- Запускается горутина, которая потребляет сообщения и логирует их.
- Если RabbitMQ недоступен — сервис работает без потребления.

2.3.3 3.3 Конфигурация

Переменная окружения `RABBITMQ_URL` добавлена в `config.go` обоих сервисов:

`RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672/`

2.3.4 3.4 Docker Compose

Добавлен контейнер `rabbitmq` с healthcheck:

```
rabbitmq:
  image: rabbitmq:3.13-management-alpine
  ports:
    - "5672:5672"
    - "15672:15672"
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "check_port_connectivity"]
```

Сервисы `booking-service` и `property-service` зависят от `rabbitmq` через `depends_on: condition: service_healthy`.

2.4 4. Проверка

1. `docker-compose up --build`
2. Зарегистрировать пользователя, создать объект, создать бронирование.
3. В логах `property-service` видно сообщение:

```
received booking event: type=booking.created booking_id=...  
property_id=... status=pending
```

4. Через Management UI (<http://localhost:15672>) можно увидеть exchange, очередь и счётчики сообщений.

2.5 5. Мониторинг RabbitMQ

Для проверки корректности работы брокера сообщений использовался RabbitMQ Management UI, доступный по адресу <http://localhost:15672>.

2.5.1 5.1 Общий обзор системы

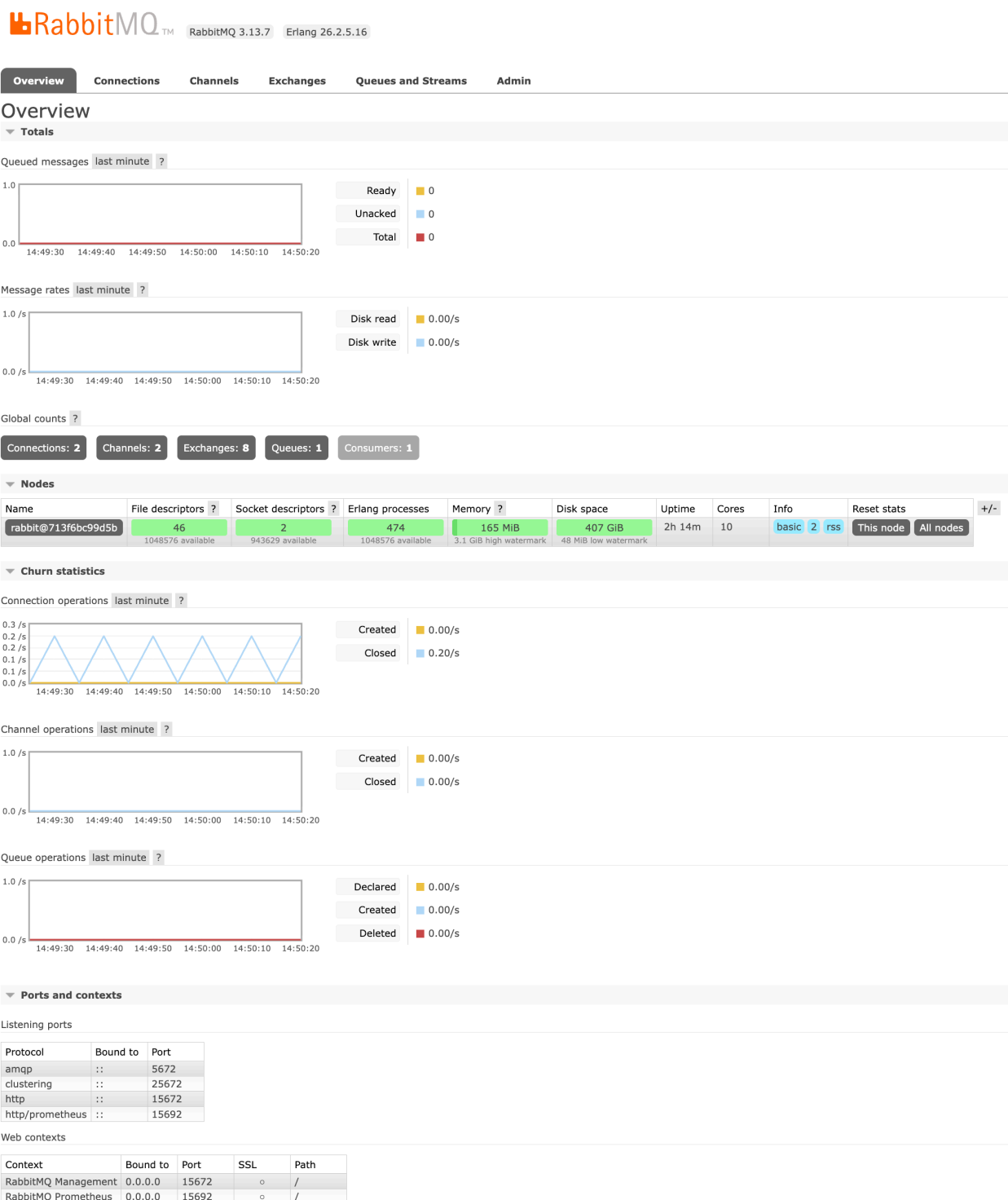


Рисунок 1 — Общий обзор RabbitMQ Management

На рисунке представлен главный экран RabbitMQ Management. Через данный интерфейс можно отслеживать количество активных соединений, каналов, очередей и обменников (exchange). Также отображается текущее состояние брокера и статистика обработки сообщений.

2.5.2 5.2 Активные соединения

Overview	Connections	Channels	Exchanges	Queues and Streams	Admin
Connections					
▼ All connections (2)					
Pagination					
Page 1 ▼ of 1 - Filter: ☐ Regex ?					
Overview		Details			Network
Name	User name	State	SSL / TLS	Protocol	Channels
172.18.0.6:45234 ?	guest	running	o	AMQP 0-9-1	1
172.18.0.7:39854 ?	guest	running	o	AMQP 0-9-1	1
					From client
					To client
					2 B/s
					2 B/s
HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub					

Рисунок 2 — Список активных соединений

На рисунке отображаются активные соединения между сервисами приложения и брокером RabbitMQ. Наличие подключений подтверждает успешную настройку сетевого взаимодействия и возможность обмена сообщениями между сервисами.

2.5.3 5.3 Каналы обмена сообщениями

RabbitMQTM

RabbitMQ 3.13.7

Erlang 26.2.5.16

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Channels

▼ All channels (2)

Pagination

Page

1 ▼

 of 1 - Filter: ☐ Regex

?

Overview				Details			Message rates					+/-
Channel	User name	Mode ?	State	Unconfirmed	Prefetch ?	Unacked	publish	confirm	unroutable (drop)	deliver / get	ack	
172.18.0.6:45234 (1)	guest		idle	0		0						
172.18.0.7:39854 (1)	guest		idle	0		0						

HTTP API

Documentation

Tutorials

New releases

Commercial edition

Commercial support

Discussions

Discord

Plugins

GitHub

Рисунок 3 — Активные каналы RabbitMQ

На рисунке представлены открытые каналы RabbitMQ. Каналы используются для публикации и потребления сообщений внутри одного соединения. Их наличие подтверждает корректную работу механизма publish/subscribe между сервисами booking-service и property-service.

3 Вывод

Подключен и настроен RabbitMQ для межсервисного взаимодействия. Реализована модель publish/subscribe: booking-service публикует события бронирований, property-service потребляет их через fanout exchange. Оба сервиса работают корректно и при недоступности брокера (graceful degradation).